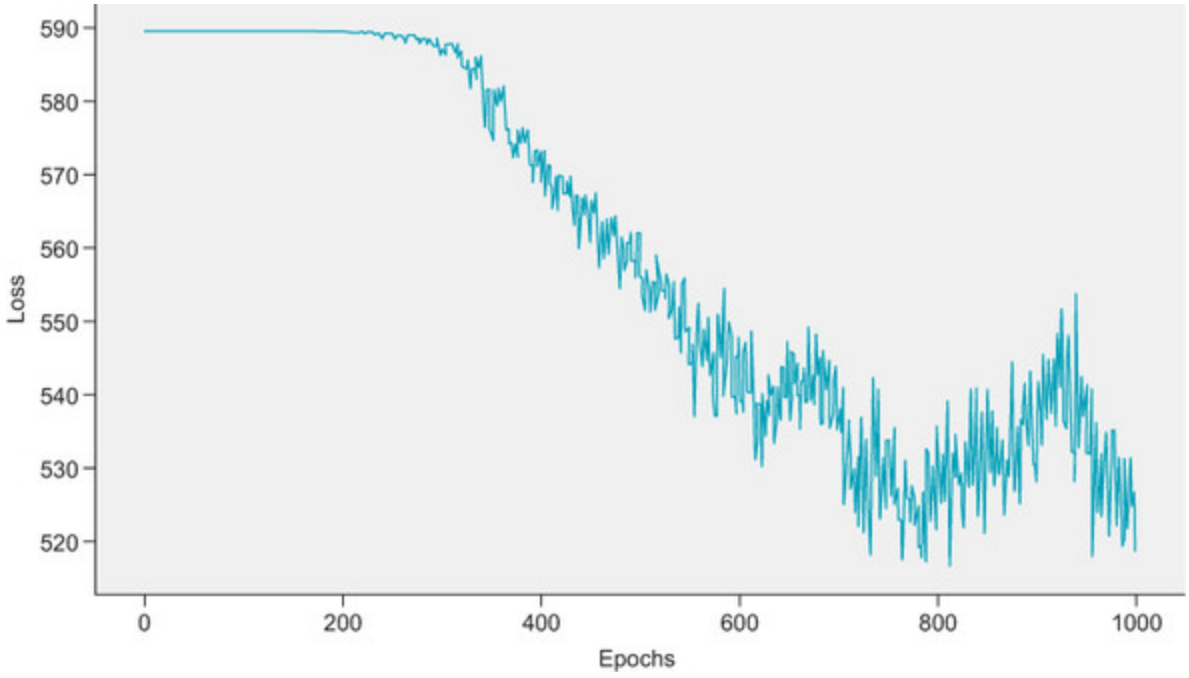
- ***7*** Adds experience to replay memory
- ***8*** If reward is 10, that indicates a successful crossing, and we want to amplify this experience.
- ***9*** Once replay buffer is full, begins training
- ***10*** Gradient descent
- ***11*** Synchronizes the target network parameters to the main model parameters
- ***12*** Decrements epsilon as a function of the epoch number
- ***13*** Resets the environment if the game is over

Almost all of this is the same kind of code we used for the ordinary DQN a few chapters ago. The only changes are that we're dealing with Q distributions rather than single Q values and that we use prioritized replay. If you plot the losses, you should get something like figure 7.20.

Figure 7.20. The loss plot for training Dist-DQN on the Atari game Freeway. The loss gradually declines but has significant "spikiness" due to the periodic target network updates.



The loss plot in figure 7.20 generally goes down but has "spikiness" due to the updates of the target network, just like we saw with the simulated example. If you investigate the `cum_rewards` list, you should get a list of ones [1, 1, 1, 1, 1, 1] indicating how many successful chicken crossings